

Vehicle Showroom Management System

Project Documentation

OOP Project C++ with Raylib GUI

1. What is this project

This is a Vehicle Showroom Management System built in C++. The idea was simple: imagine a real car showroom where different people come in — customers who want to buy a vehicle, salesmen who handle the sales, and repairmen who fix vehicles after purchase. The admin runs the whole place. This program simulates all of that.

Instead of being a boring terminal-only program, it actually has a graphical window with buttons and input fields, built using a library called Raylib. So you log in, click around, buy vehicles, request repairs — all through an actual UI.

The whole thing is written as one big C++ file and uses file storage (plain .txt files) to save all data, since there is no database. Every time you close the program, it writes everything to those text files and loads them back when you open it again.

2. Why was it built

This was built as an OOP (Object Oriented Programming) course project. The goal was to practice and demonstrate core OOP concepts — inheritance, polymorphism, encapsulation, and abstraction — in a real scenario rather than just toy examples.

The showroom theme was picked because it maps very cleanly to OOP. You have clear hierarchies: all vehicles share common stuff, all people share common stuff, and then each type gets its own extra things. It is also close to something real, which makes it easier to explain what each class is doing and why.

3. Tools and technologies used

Language	C++ (using STL — vectors, strings, fstream, sstream, ctime)
Graphics	Raylib — a simple C library for making windows, buttons, text input etc.
Storage	Plain .txt files with pipe () separated values. No database.
Font	Roboto-Regular.ttf loaded via Raylib's LoadFontEx
Build	Standard C++ compiler. Raylib needs to be linked separately.

Raylib handles all the window drawing, mouse clicks, keyboard input, and button hover effects. It is normally used for games but works great for this kind of simple GUI too. Every frame the program redraws the whole screen at 60 FPS.

4. Class hierarchy overview

There are two big inheritance trees in this project — one for vehicles and one for people. Then there are a bunch of manager/helper classes sitting on the side that handle the business logic.

Vehicle tree

Vehicle is the base class. Car, Bike, and Scooty all inherit from it. Vehicle holds everything that every vehicle has — model ID, brand, model name, year, color, price, quantity, whether it is electric, engine CC, mileage, and fuel type. It has two pure virtual methods: `displayInfo()` and `getType()`, which force every subclass to define its own version of those.

Vehicle	Abstract base. Has all the shared vehicle fields. Cannot be instantiated on its own.
Car	Adds: seating capacity, transmission type (manual/auto), number of doors.
Bike	Adds: bike type (sports, cruiser, standard, etc.).
Scooty	Adds: storage capacity in litres (the under-seat box).

Person tree

Person is the abstract base for any human in the system. It holds id, name, email, phone, username, and password. There are pure virtual methods `showMenu()`, `getRole()`, and `disp_Info()` — so every subclass has to implement them.

Employee is a middle class that extends Person and adds CNIC, hire date, and salary. Both Salesman and RepairMan extend Employee since they are both staff.

Customer extends Person directly (not Employee) since customers are not staff — they get extra fields like address, CNIC, and registration date instead.

Person	Abstract base. Handles identity and login credentials.
Employee	Extends Person. Adds employment fields: CNIC, hire date, salary.
Customer	Extends Person. Adds: CNIC, address, registration date.
Salesman	Extends Employee. Adds: sales count. Can sell vehicles.
RepairMan	Extends Employee. Adds: specialization. Can accept and complete repair jobs.

There is also an Admin class. Admin extends Employee and has full control — adding/removing vehicles, registering salesmen, managing repairmen, viewing all records, and adjusting salaries.

5. Classes explained one by one

Vehicle

This is the root of the vehicle family. It is declared as an abstract class because `displayInfo()` and `getType()` are pure virtual. That means you can never do 'Vehicle v;' — you can only create a Car, Bike, or Scooty. But you can have a pointer of type `Vehicle*` pointing to any of those, which is how the inventory returns mixed lists of vehicles.

It has a default constructor that fills everything with zeroes and NULLs, a parameterized constructor, and a copy constructor. The destructor is virtual so that when you delete through a base pointer the right destructor runs.

Key methods: `get_model_id`, `get_brand`, `get_price`, `get_quantity`, `reduceQuantity` (throws if stock goes below 0), `increaseQuantity`, `setPrice`, `setQuantity`.

Car (extends Vehicle)

Car adds three things on top of Vehicle: seating capacity (how many people fit), transmission (Manual or Automatic), and number of doors. Everything else is inherited.

It also has a static method called `getCarModel(modelID)` which opens `cars.txt`, reads through it line by line, and returns a heap-allocated Car pointer if it finds a match. The file stores each field separated by pipe characters.

Bike (extends Vehicle)

Bike adds a `bikeType` field — things like Sports, Cruiser, Standard, Off-Road. Same pattern as Car: has its own static `getBikeModel(modelID)` that reads `bikes.txt`.

Scooty (extends Vehicle)

Scooty adds `storageCapacityLiters` which represents the under-seat storage compartment. Has its own `getScootyModel()` static method reading from `scooties.txt`.

Inventory

This class does not extend anything. It is just a container that holds three vectors: one for cars, one for bikes, one for scooties. It is the showroom's stock management.

loadAll() reads all three text files and populates the vectors when the program starts. saveAll() writes them all back. Every time a sale happens or stock changes, saveAll() is called so nothing is lost.

getAllVehicles() returns a vector of Vehicle* pointers — mixing all three types together into one list. This works because of polymorphism: Car, Bike, and Scooty all are Vehicles.

findByModelID(id) loops through all three vectors and returns a Vehicle* to whatever matches. removeModel(id) similarly scans all three and erases the right one. updateQuantity(id, n) uses findByModelID first, then calls setQuantity.

Person

Abstract base class for all humans in the system. Has id, name, email, phone, username, and password. checkPassword(input) just compares the stored string to what the user typed — no hashing, plain text passwords (fair enough for a university project).

The three pure virtual methods — showMenu(), getRole(), disp_Info() — mean that the Showroom can hold a Person* and call those methods without knowing if it is a Customer, Salesman, or RepairMan at that point. That is polymorphism in action.

Employee (extends Person)

A middle layer between Person and the staff classes. Adds CNIC, hire date, and salary. setSalary() throws an error if you try to set a negative salary. Salesman and RepairMan both inherit from Employee.

Customer (extends Person)

Customers are not employees so they extend Person directly. Extra fields: CNIC, address, and registration date.

What a customer can do: browse cars, bikes, or scooties (just calls Inventory's display method), view their own purchase history, view their repair history, and request a repair for one of their vehicles.

requestVehicleRepair() gets the current date using C's time() functions (the comment in code says this was taken from online resources), then forwards the request to RepairManager.

getCustomer_ById() is a static method that opens customer.txt and finds a customer by their ID. Same file-reading pattern as the vehicle classes.

Salesman (extends Employee)

Salesman adds a sales count (how many vehicles this person has sold). `incrementSalesCount()` ticks it up by one every time a sale happens.

The main function is `sellVehicle(inv, cust, sm, modelID)`. Here is what it does step by step: finds the vehicle in inventory, reduces its quantity by 1, generates a random plate number, gets the current date, creates a `SaleRecord` and a `SoldVehicle` object, records them in `SalesManager`, then saves everything to files right away.

`getSalesCount()` and `viewMySalesHistory()` let the salesman see how they are doing. `getSalesman_ById()` is the static file-reading method same as before.

RepairMan (extends Employee)

RepairMan adds a specialization field (like Engine, Electrical, Body Work etc.). A repairman can view pending jobs in the system, accept a specific job by its repair ID, and mark a job as completed.

`acceptJob(repairManager, repairID)` calls `RepairManager's assignRepairMan` and sets the `RepairManID` on that record to this person's ID. `markJobCompleted()` calls `updateStatus` on the repair manager passing 'Completed'.

Admin (extends Employee)

Admin is the superuser. Extends `Employee` and has access to basically everything. Through the GUI the admin can: add new cars, bikes, or scooties to inventory, remove vehicles, update prices and quantities, register new salesmen, register new repairmen, fire (remove) a salesman or repairman, update their salaries, view all sales records, view total revenue, and view all repair records.

Admin holds references/pointers to the `Inventory`, `SalesManager`, and `RepairManager` so it can coordinate across all of them.

SaleRecord

`SaleRecord` stores a single transaction. It has a sale ID (like SL001, SL002...), model ID, vehicle type, customer ID, salesman ID, date, and price. The IDs are generated by just counting how many records already exist and adding one.

`getSaleRecord(saleID)` is the static method to look one up from `sales.txt`. `dispRec()` prints all the fields in a formatted way.

SoldVehicle

SoldVehicle is similar to SaleRecord but stores the actual vehicle-specific info for the customer's perspective — plate number, model ID, type, brand, model name, customer ID, salesman ID, sale date, and price. It is what shows up when a customer checks their purchase history.

Every successful sale creates both a SaleRecord (salesman's view) and a SoldVehicle (customer's vehicle record). Both get saved into separate text files.

SalesManager

SalesManager holds two vectors: one for SaleRecord objects and one for SoldVehicle objects. It loads from sales.txt and sold_Vehicles.txt on startup, and saves back to them when something changes.

Key methods: record_Sale() adds a new pair (SaleRecord + SoldVehicle) to both vectors. get_Purchases_By_Customer(id) filters the soldVehicles list by customer ID. get_Sales_By_Salesman(id) filters the sales list by salesman ID. get_Total_Revenue() sums up all sale prices. generate_PlateNumber() produces a random 3-letter-dash-4-digit plate like ABC-1234.

RepairRecord

RepairRecord stores a repair job. Fields: repair ID (RP001, RP002...), customer ID, plate number of the vehicle being repaired, vehicle type, part being changed, cost, repairman ID (starts as NULL until someone accepts the job), status (Pending or Completed), and the date the request was made.

There is a variable in here called cost___chunna — the triple underscore and the word chunna is clearly a little joke left in by the developer. It still works fine.

PartPrice and RepairManager

PartPrice is a tiny class that just holds a part name and its cost. It is used by RepairManager to look up prices when a customer requests a repair.

RepairManager holds a vector of RepairRecord objects and a vector of PartPrice objects. It loads from repairs.txt and Price_list_of_Parts.txt. requestRepair() looks up the part price, generates a new repair ID, creates a new RepairRecord with Pending status, and adds it to the list. getPendingJobs() returns only the ones still marked Pending. assignRepairMan() links

a repairman to a job. `updateStatus()` changes the status.

ShowroomSystem

This is the top-level class that ties everything together. It holds: an Inventory, a SalesManager, a RepairManager, and vectors of Salesman, Customer, and RepairMan objects. It also holds a pointer to the currently logged-in Person.

The `run()` method is where the main loop lives. It uses Raylib's `BeginDrawing / EndDrawing` loop running at 60 FPS. It tracks which screen to show using an enum (MAIN for login, then CUSTOMER_MENU, SALESMAN_MENU, ADMIN_MENU, REPAIRMAN_MENU).

`authenticateLogin(username, password)` checks the input against all three employee lists (salesmen, customers, repairmen) and returns a `Person*` if found. `saveEverything()` writes salesmen, customers, and repairmen back to their files. It is called when anything changes.

6. How the GUI works

The GUI is built on top of Raylib. There are three small structs that handle the reusable UI pieces: `InputBox`, `Button`, and a couple of helper functions.

InputBox

`InputBox` draws a labelled text field. When it is focused (clicked on) it shows a blinking cursor and highlights the border in blue. It handles typing via `GetCharPressed()` and backspace via `IsKeyPressed(KEY_BACKSPACE)`. If `password=true` it shows stars instead of actual characters.

Button

`Button` draws a rectangle with centred text. When the mouse hovers over it, the background gets lighter. When clicked it goes blue. `draw()` returns true in the frame that the mouse button is released over it — so you just do 'if (btn.draw())' to handle a click.

Modals and notifications

`drawFormModal()` draws a centred dark overlay with a title, a set of `InputBox` fields, and Confirm/Cancel buttons. It is used everywhere something needs input — selling a vehicle, accepting a job, adding inventory, etc.

`showNotif()` and `drawNotif()` show a bar at the bottom of the screen for 2.5 seconds in green for success or red for errors. This is how the user knows if something worked or failed.

`drawInfoList()` is used for scrollable lists — it uses Raylib's scissor mode to clip content to a rectangle, and handles mouse wheel scrolling.

7. How file storage works

Everything is stored in plain text files. Each file starts with a header line and then one record per line. Fields within a line are separated by pipe characters (`|`). This makes it easy to read with `getline(..., '|')` in C++.

<code>cars.txt</code>	One car per line. Fields: <code>modelID</code> , <code>brand</code> , <code>model</code> , <code>year</code> , <code>color</code> , <code>price</code> , <code>qty</code> , <code>electric</code> , <code>CC</code> , <code>mileage</code> , <code>fuel</code> , <code>seats</code> , <code>transmission</code> , <code>doors</code> .
<code>bikes.txt</code>	Same structure but ends with <code>bikeType</code> instead of the car-specific fields.
<code>scooties.txt</code>	Same structure, ends with storage capacity in litres.
<code>sales.txt</code>	One sale per line. Fields: <code>saleID</code> , <code>modelID</code> , <code>vehicleType</code> , <code>customerID</code> , <code>salesmanID</code> , <code>date</code> , <code>price</code> .
<code>sold_Vehicles.txt</code>	One sold vehicle per line. Fields: <code>plateNumber</code> , <code>modelID</code> , <code>type</code> , <code>brand</code> , <code>modelName</code> , <code>customerID</code> , <code>salesmanID</code> , <code>date</code> , <code>price</code> .
<code>salesman.txt</code>	One salesman per line. Fields: <code>id</code> , <code>name</code> , <code>email</code> , <code>phone</code> , <code>CNIC</code> , <code>hireDate</code> , <code>salary</code> , <code>salesCount</code> , <code>username</code> , <code>password</code> .
<code>customer.txt</code>	One customer per line. Fields: <code>id</code> , <code>name</code> , <code>email</code> , <code>phone</code> , <code>CNIC</code> , <code>address</code> , <code>regDate</code> , <code>username</code> , <code>password</code> .
<code>repairman.txt</code>	One repairman per line. Fields: <code>id</code> , <code>name</code> , <code>email</code> , <code>phone</code> , <code>CNIC</code> , <code>hireDate</code> , <code>salary</code> , <code>specialization</code> , <code>username</code> , <code>password</code> .
<code>repairs.txt</code>	One repair job per line. Fields: <code>repairID</code> , <code>customerID</code> , <code>plateNumber</code> , <code>vehicleType</code> , <code>partChanged</code> , <code>cost</code> , <code>repairManID</code> , <code>status</code> , <code>date</code> .
<code>Price_list_of_Parts.txt</code>	Part name and cost per line.

The `number_of_models()` utility function counts lines in a file to know how many records to read. It subtracts 2 to account for the header and a trailing newline. Every class has a `loadAll()` and `saveAll()` pair — load on startup, save after every change.

8. How the classes relate to each other

Here is a plain-English walkthrough of who knows about who and why.

Inheritance (is-a relationships)

Car, Bike, Scooty inherit from Vehicle — they are all types of Vehicle.

Employee inherits from Person — an employee is a person.

Customer inherits from Person — a customer is a person.

Salesman inherits from Employee — a salesman is an employee.

RepairMan inherits from Employee — a repairman is an employee.

Admin inherits from Employee — an admin is an employee.

Composition / has-a relationships

Inventory has vectors of Car, Bike, and Scooty.

SalesManager has vectors of SaleRecord and SoldVehicle.

RepairManager has vectors of RepairRecord and PartPrice.

ShowroomSystem has an Inventory, a SalesManager, a RepairManager, and vectors of Salesman, Customer, RepairMan.

Usage / depends-on relationships

Salesman.sellVehicle() takes an Inventory, a Customer, and a SalesManager as parameters. It updates the inventory quantity, creates sale records, and tells the SalesManager to save.

Customer.requestVehicleRepair() takes a RepairManager and forwards the request. Customer.viewMyPurchases() takes a SalesManager and filters by their ID.

RepairMan.acceptJob() and markJobCompleted() take a RepairManager and update records inside it.

ShowroomSystem.authenticateLogin() checks all three person vectors and returns a Person* — the GUI then casts it to the right type based on getRole() to show the right menu.

9. OOP concepts used and where

Inheritance	Vehicle > Car/Bike/Scooty. Person > Employee > Salesman/RepairMan/Admin. Person > Customer.
Abstraction	Vehicle and Person are abstract (have pure virtual methods). You can never create them directly.
Polymorphism	Inventory stores Vehicle* pointers. ShowroomSystem holds a Person* for whoever is logged in. displayInfo() and getRole() behave differently for each subclass.
Encapsulation	All fields are private or protected. Access is only through getters and setters. Setters validate input (e.g. setSalary rejects negatives, reduceQuantity throws if stock goes below 0).
Copy constructor	Every class has one to make sure deep copies work when pushing into vectors.
Virtual destructor	Vehicle and Person have virtual destructors so cleanup works correctly through base pointers.
Static methods	getCarModel(), getSalesman_ById(), getCustomer_ById() etc. are static — they work without needing an object instance.

10. What happens during a sale (step by step)

This is what actually runs when a salesman sells a vehicle through the UI:

Step 1 — Salesman logs in with username and password. ShowroomSystem checks the salesmen vector, returns a Salesman* if found.

Step 2 — The GUI switches to SALESMAN_MENU. Salesman sees buttons including 'Sell Vehicle'.

Step 3 — A modal pops up asking for the customer ID and the model ID.

Step 4 — Customer is looked up from the customers vector.

Step 5 — sellVehicle(inv, cust, sm, modelID) is called on the Salesman object.

Step 6 — Inventory.findByModelID() finds the Vehicle* and checks quantity.

Step 7 — Inventory.updateQuantity() reduces stock by 1.

Step 8 — SalesManager.generate_PlateNumber() creates a random plate like 'KXR-4821'.

Step 9 — Current date is fetched using localtime().

Step 10 — A SaleRecord and a SoldVehicle are created and added to SalesManager.

Step 11 — SalesManager.saveAll() writes sales.txt and sold_Vehicles.txt.

Step 12 — Inventory.saveAll() writes updated quantities back to the vehicle files.